

**GUIA DE CONCEITOS AVANÇADOS DE DESENVOLVIMENTO
FRONT-END: HTML5 SEMÂNTICO, ARQUITETURA CSS E
ACESSIBILIDADE**

1. SEPARAÇÃO E RESPONSABILIDADE DOS ARQUIVOS CSS

Pergunta: Por que devemos dividir o nosso código CSS em múltiplos arquivos em vez de manter tudo em um único arquivo?

Resposta: A separação de arquivos CSS aplica o princípio da responsabilidade única na arquitetura de software. Cada arquivo assume um papel específico na aplicação (como reset de estilos padrão, definições do sistema de design ou estilização de componentes específicos). Isso facilita a manutenção, previne efeitos colaterais indesejados e torna o código reutilizável e legível para grandes equipes.

```
/* Estrutura de Diretórios e Arquivos Modularizados */  
  
├─ reset.css /* Reseta padrões dos navegadores */  
├─ design-system.css /* Variáveis globais e paletas */  
├─ colors-bars.css /* Estilos das barras de cores */  
└─ global.css /* Centralizador e gerenciador de importações */
```

2. A IMPORTÂNCIA DA SEQUÊNCIA DE IMPORTAÇÃO NO CSS

Pergunta: Qual é o impacto da ordem das instruções @import dentro do arquivo CSS principal?

Resposta: O CSS opera com base nos conceitos de cascata, especificidade e ordem de declaração. O navegador processa as regras de cima para baixo. Portanto, arquivos de base (reset) devem ser carregados primeiro, seguidos pelas variáveis globais e, por fim, pelos estilos dos componentes específicos. Se a ordem for invertida, regras genéricas de reset podem sobrescrever estilizações personalizadas já definidas.

```
/* Arquivo: global.css - A ordem correta garante a cascata adequada */  
@import url(./reset.css); /* 1º: Base limpa */  
@import url(./design-system.css); /* 2º: Variáveis globais (:root) */  
@import url(./colors-bars.css); /* 3º: Componentes que usam as variáveis */
```

3. USO SEMÂNTICO DA DIV VERSUS A TAG SECTION

Pergunta: Quando devemos utilizar a tag <section> e quando é mais adequado utilizar uma <div>?

Resposta: A tag <section> representa um agrupamento temático de conteúdo que possui significado próprio dentro da árvore do documento e normalmente contém um título. Já a <div> é um container genérico sem valor semântico, devendo ser utilizada exclusivamente para fins de estrutura de layout no CSS (como envolver elementos para aplicar propriedades Flexbox ou Grid).

```
<!-- Estrutura Semântica Recomendada -->  
<section id="rgb" class="color-section"> <aside  
class="color-section__sidebar">...</aside>
```

```
<!-- Div utilizada apenas para envolver o layout flexível (container  
neutro) --> <div class="color-section__content">  
<section class="color-section__palette">...</section>
```

```
<!-- Subseção da Paleta --> <section  
class="color-section__faq">...</section> <!-- Subseção de FAQ --> </div>  
</section>
```

4. ACESSIBILIDADE COM ATRIBUTOS ARIA (ARIA-LABEL E ARIA-LABELLEDBY)

Pergunta: De que forma os atributos aria-label e aria-labelledby melhoram a acessibilidade da página?

Resposta: Esses atributos fornecem rótulos descritivos para tecnologias assistivas (como leitores de tela). O atributo aria-label define diretamente um texto descritivo para o elemento, enquanto o aria-labelledby associa uma seção ao ID de um título já existente na página, criando marcos de navegação claros.

```
<!-- Rotulagem direta para leitores de tela -->
<section class="color-section__palette" aria-label="Visualização da Paleta RGB">
<!-- Rotulagem vinculada ao ID do título da subseção -->
<section class="color-section__faq" aria-labelledby="faq-rgb-title"> <h3
```

5. COMPONENTES INTERATIVOS NATIVOS: <DETAILS> E <SUMMARY>

Pergunta: Como criar um componente de expansão (accordion/FAQ) nativo sem utilizar JavaScript?

Resposta: O HTML5 disponibiliza as tags nativas <details> e <summary>. O elemento <summary> atua como o cabeçalho clicável, enquanto o conteúdo subsequente dentro de <details> é exibido ou ocultado automaticamente pelo navegador ao interagir com o componente.

```
<details class="color-section__faq-item">
<summary class="color-section__faq-question">O que significa a sigla RGB?</summary> <p class="color-section__faq-answer">Significa Red, Green e Blue (Vermelho, Verde e Azul).</p> </details>
```

6. SELETORES DE ATRIBUTO PADRÃO NO CSS ([ATRIBUTO])

Pergunta: Como estilizar elementos no CSS com base em seus atributos HTML ou estados dinâmicos?

Resposta: Utilizamos os seletores de atributo marcados por colchetes [atributo]. No caso do componente <details>, quando aberto, o HTML recebe automaticamente o atributo booleano "open". Podemos capturar esse estado no CSS para alterar a aparência e aplicar efeitos visuais de destaque.

```
/* Estilização aplicada somente quando o elemento tiver o atributo 'open' */  
.  
color-section__faq-item[open]  
{ background-color: var(--faq-card-hover-bg); border-color:  
var(--faq-border-neon); box-shadow: 0 0 8px var(--faq-border-neon), 0 0  
20px rgba(157, 38, 239, 0.3); }
```

7. APRIMORAMENTO VISUAL COM PSEUDO-ELEMENTOS (::AFTER) E ESTILIZAÇÃO DE CABEÇALHOS

Pergunta: Como remover o ícone de seta padrão da tag <summary> e inserir um ícone customizado com animação via CSS?

Resposta: Removemos o marcador nativo usando `list-style: none` no <summary> e ocultando `::-webkit-details-marker`. Em seguida, inserimos um novo símbolo com o pseudo-elemento `::after` e aplicamos transformações de rotação quando o pai <details> possuir a classe ou atributo [open].

```
/* Oculta o marcador padrão */
.color-section__faq-question
{ list-style: none; display: flex; justify-content: space-between; }
.color-section__faq-question::-webkit-details-marker { display: none; }

/* Adicionar ícone customizado */
.color-section__faq-question::after { content: "◆"; transition: transform
0.3s ease; } /* Rotaciona o ícone ao abrir */
.color-section__faq-item[open] .color-section__faq-question::after {
transform: rotate(180deg) scale(1.3); }
```

8. APLICAÇÕES DE PSEUDO-CLASSES E PSEUDO-ELEMENTOS INTERATIVOS

Pergunta: Qual a diferença entre pseudo-classes e pseudo-elementos e como eles foram aplicados nos estados de hover?

Resposta: Pseudo-classes (como `:hover` e `:not`) definem o estilo de um elemento baseado em seu estado dinâmico ou posição na árvore. Pseudo-elementos (como `::after` e `::before`) permitem estilizar partes específicas de um elemento ou inserir conteúdo decorativo. Utilizamos `:hover` nos cards para alterar cores e sombras suavemente.

```
/* Mudança de cor no hover da resposta */  
.color-section__faq-answer:hover { color: #ffffff; }  
  
/* Destaque nos links do menu que nao estao ativos */  
.color-section__link:not(.color-section__link--active):hover {  
background-color: var(--ui-link-hover-bg); }
```

9. UNIDADES DE MEDIDA RELATIVAS AO VIEWPORT (VH) PARA CONTROLE DE LAYOUT

Pergunta: Por que definimos a altura da seção de cores em 43vh?

Resposta: A unidade "vh" (viewport height) é relativa a 1% da altura da tela visível. Ao fixar o container superior das barras em 43vh, garantimos que os 57vh restantes sejam suficientes para acomodar a seção do FAQ e o conteúdo expandido das perguntas sem provocar o aparecimento da barra de rolagem na janela principal.

```
.color-section__palette
{ display: flex; height: 43vh;
/* Reservado 43% da altura total da tela */
width: 100%; }
.color-section__faq { flex-grow: 1;
/* Preenche os 57vh restantes sem estourar o viewport */ }
```